

06 prefinality.py - Clear before finality

Source: *gate/prefinality.py*

```
"""Pre-finality clearance - GO/NO-GO before irreversible commit (x402, RTP).

Rail-agnostic evaluate contract:
    POST body -> decision + signed JWT receipt bound to transfer fingerprint.

Fail-closed: missing fields, policy breach, fuse DEAD, or unsigned receipt in prod -> NO_GO.
"""
from __future__ import annotations

import base64
import hashlib
import json
import os
import re
import secrets
import uuid
from datetime import datetime, timedelta, timezone
from typing import Any, Callable

try:
    from gate import receipt as receipt_mod
except ImportError:
    import receipt as receipt_mod

SPEC = "gate-prefinality-v1"
RAILS = ("x402", "rtp")
DECISIONS = ("GO", "NO_GO", "HOLD")
DEFAULT_TTL_SECONDS = 300

_EVM_RE = re.compile(r"^[a-fA-F0-9]{40}$")
_ROUTING_RE = re.compile(r"^[d]{9}$")
_ACCOUNT_RE = re.compile(r"^[d]{4,17}$")

def _dev_mode() -> bool:
    return os.getenv("GATE_DEV_MODE", "").strip().lower() in ("1", "true", "yes", "on")

def _utc_now() -> datetime:
    return datetime.now(timezone.utc)

def _iso(dt: datetime) -> str:
    return dt.replace(microsecond=0).isoformat()

def _b64url(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode("ascii")

def _b64url_decode(s: str) -> bytes:
    pad = "=" * (-len(s) % 4)
    return base64.urlsafe_b64decode(s + pad)

def _canonical_json(obj: Any) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

def signing_required() -> bool:
    return receipt_mod.signing_required()

def _signing_key():
    return receipt_mod._ed25519_signing_key()

def _public_key_bytes() -> bytes | None:
    return receipt_mod._ed25519_public_key_bytes()
```

```

def key_id() -> str | None:
    fp = receipt_mod.receipt_public_key_fingerprint()
    return f"gate-prefinality-{{fp}}" if fp else None

def transfer_fingerprint(*, rail: str, transfer: dict) -> str:
    body = {"spec": SPEC, "rail": rail, "transfer": _normalize_transfer(rail, transfer)}
    return hashlib.sha256(_canonical_json(body).encode("utf-8")).hexdigest()

def _normalize_transfer(rail: str, transfer: dict | None) -> dict:
    t = transfer if isinstance(transfer, dict) else {}
    out: dict[str, Any] = {}
    amount = t.get("amount")
    if amount is not None:
        out["amount"] = str(amount).strip()
    currency = (t.get("currency") or "").strip().upper()
    if currency:
        out["currency"] = currency
    cp = (t.get("counterparty") or t.get("payto") or t.get("payTo") or "").strip()
    if cp:
        out["counterparty"] = cp
    if rail == "rtp":
        routing = (t.get("routing_number") or "").strip()
        account = (t.get("account_number") or "").strip()
        if routing:
            out["routing_number"] = routing
        if account:
            out["account_number"] = _mask_account(account)
        ext = (t.get("external_account_id") or "").strip()
        if ext:
            out["external_account_id"] = ext
    resource = (t.get("resource_url") or "").strip()
    if resource:
        out["resource_url"] = resource[:512]
    return out

def _mask_account(account: str) -> str:
    digits = re.sub(r"\D", "", account)
    if len(digits) <= 4:
        return "****"
    return f"****{digits[-4:]}"

def _parse_amount(raw) -> float | None:
    if raw is None:
        return None
    try:
        return float(str(raw).strip())
    except (TypeError, ValueError):
        return None

def _validate_transfer(rail: str, transfer: dict) -> list[str]:
    errors: list[str] = []
    t = _normalize_transfer(rail, transfer if isinstance(transfer, dict) else {})
    amount = _parse_amount(t.get("amount"))
    if amount is None or amount <= 0:
        errors.append("invalid_amount")
    currency = (t.get("currency") or "").upper()
    cp = t.get("counterparty") or ""
    if rail == "x402":
        if currency not in ("", "USDC", "USD"):
            errors.append("unsupported_currency")
        if not _EVM_RE.match(cp):
            errors.append("invalid_counterparty")
    elif rail == "rtp":
        if currency not in ("", "USD"):
            errors.append("unsupported_currency")
        has_ext = bool(t.get("external_account_id"))
        has_bank = bool(transfer.get("routing_number") and transfer.get("account_number"))
        if not has_ext and not has_bank:

```

```

        errors.append("rtp_counterparty_required")
    if transfer.get("routing_number") and not _ROUTING_RE.match(str(transfer["routing_number"]).strip()):
        errors.append("invalid_routing_number")
    if transfer.get("account_number") and not _ACCOUNT_RE.match(re.sub(r"\D", "", str(transfer["account_number"]))):
        errors.append("invalid_account_number")
    return errors

```

```

def _policy_signals(
    *,
    rail: str,
    transfer: dict,
    mandate: dict | None,
    context: dict | None,
) -> tuple[str, list[str]]:
    """Return decision + signal codes."""
    signals: list[str] = []
    mandate = mandate if isinstance(mandate, dict) else {}
    context = context if isinstance(context, dict) else {}
    t = transfer if isinstance(transfer, dict) else {}

    errors = _validate_transfer(rail, t)
    if errors:
        return "NO_GO", errors

    amount = _parse_amount(t.get("amount"))
    max_amount = _parse_amount(mandate.get("max_amount") or mandate.get("max_payment"))
    if max_amount is not None and amount is not None and amount > max_amount:
        signals.append("amount_exceeds_cap")
        return "NO_GO", signals

    expected = (
        mandate.get("expected_payto")
        or mandate.get("expected_counterparty")
        or mandate.get("expected_payto")
        or ""
    ).strip()
    actual = (t.get("counterparty") or t.get("payto") or t.get("payto") or "").strip()
    if expected and actual and expected.lower() != actual.lower():
        signals.append("routing_anomaly")
        return "NO_GO", signals

    untrusted = (context.get("untrusted_text") or context.get("untrusted") or "").strip()
    if untrusted and actual and actual.lower() in untrusted.lower():
        signals.append("injection_destination")
        return "NO_GO", signals

    intent = (mandate.get("intent") or context.get("intended") or "").strip()
    if intent and untrusted and intent.lower() not in untrusted.lower() and actual:
        # Soft signal only when intent is declared but doesn't match untrusted source.
        if untrusted.lower() in actual.lower():
            signals.append("intent_mismatch")
            return "NO_GO", signals

    review_ceiling = _parse_amount(mandate.get("review_ceiling") or mandate.get("hold_above"))
    if review_ceiling is not None and amount is not None and amount > review_ceiling:
        signals.append("review_threshold")
        return "HOLD", signals

    daily_cap = _parse_amount(mandate.get("daily_cap"))
    if daily_cap is not None and amount is not None and amount > daily_cap:
        signals.append("daily_cap_exceeded")
        return "NO_GO", signals

    return "GO", signals

```

```

def mint_receipt_jwt(
    *,
    evaluation_id: str,
    rail: str,
    decision: str,

```

```

    fingerprint: str,
    signals: list[str],
    agent_id: str | None,
    ttl_seconds: int = DEFAULT_TTL_SECONDS,
    issuer: str | None = None,
) -> str | None:
    key = _signing_key()
    if not key:
        return None
    now = _utc_now()
    exp = now + timedelta(seconds=max(30, min(int(ttl_seconds or DEFAULT_TTL_SECONDS), 3600)))
    kid = key_id()
    payload = {
        "spec": SPEC,
        "jti": evaluation_id,
        "iss": issuer or "gate.velaru.xyz",
        "sub": agent_id or "anonymous",
        "iat": int(now.timestamp()),
        "exp": int(exp.timestamp()),
        "rail": rail,
        "dec": decision,
        "fp": fingerprint,
        "sig": signals,
    }
    header = {"alg": "EdDSA", "typ": "JWT", "kid": kid}
    header_b64 = _b64url(_canonical_json(header).encode("utf-8"))
    payload_b64 = _b64url(_canonical_json(payload).encode("utf-8"))
    signing_input = f"{header_b64}.{payload_b64}".encode("utf-8")
    sig = key.sign(signing_input)
    return f"{header_b64}.{payload_b64}.{_b64url(sig)}"

```

```

def verify_receipt_jwt(token: str, *, expected_fingerprint: str | None = None) -> dict:

```

```

    meta = {
        "spec": SPEC,
        "valid": False,
        "decision": None,
        "reason": None,
        "payload": None,
    }
    if not token or not isinstance(token, str):
        meta["reason"] = "missing_token"
        return meta
    parts = token.split(".")
    if len(parts) != 3:
        meta["reason"] = "malformed_jwt"
        return meta
    try:
        header = json.loads(_b64url_decode(parts[0]))
        payload = json.loads(_b64url_decode(parts[1]))
        sig = _b64url_decode(parts[2])
    except Exception:
        meta["reason"] = "decode_error"
        return meta

    pub = _public_key_bytes()
    if not pub:
        meta["reason"] = "verify_unconfigured"
        return meta
    try:
        from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PublicKey

        pk = Ed25519PublicKey.from_public_bytes(pub)
        signing_input = f"{parts[0]}.{parts[1]}".encode("utf-8")
        pk.verify(sig, signing_input)
    except Exception:
        meta["reason"] = "bad_signature"
        return meta

    if header.get("alg") != "EdDSA":
        meta["reason"] = "unsupported_alg"
        return meta

```

```

exp = payload.get("exp")
if exp is not None and int(exp) < int(_utc_now().timestamp()):
    meta["reason"] = "expired"
    meta["payload"] = payload
    return meta

fp = payload.get("fp")
if expected_fingerprint and fp != expected_fingerprint:
    meta["reason"] = "fingerprint_mismatch"
    meta["payload"] = payload
    return meta

decision = (payload.get("dec") or "").upper()
if decision not in DECISIONS:
    meta["reason"] = "invalid_decision"
    meta["payload"] = payload
    return meta

meta.update({"valid": True, "decision": decision, "payload": payload, "reason": None})
return meta

def evaluate(
    body: dict,
    *,
    account_id: str | None = None,
    public_url: str,
    fuse_hop: Callable[[str], dict | None] | None = None,
) -> dict:
    """Core evaluate - returns API response dict."""
    rail = (body.get("rail") or "").strip().lower()
    transfer = body.get("transfer") if isinstance(body.get("transfer"), dict) else {}
    mandate = body.get("mandate") if isinstance(body.get("mandate"), dict) else {}
    context = body.get("context") if isinstance(body.get("context"), dict) else {}
    ttl = int(body.get("ttl_seconds") or mandate.get("ttl_seconds") or DEFAULT_TTL_SECONDS)
    agent_id = (mandate.get("agent_id") or body.get("agent_id") or context.get("agent_id") or "").strip() or None
    fuse_id = (mandate.get("fuse_id") or body.get("fuse_id") or "").strip() or None

    evaluation_id = f"pf_{uuid.uuid4().hex}"
    created_at = _iso(_utc_now())

    if rail not in RAILS:
        return _response(
            evaluation_id=evaluation_id,
            rail=rail or "unknown",
            decision="NO_GO",
            fingerprint="",
            signals=["unsupported_rail"],
            receipt=None,
            created_at=created_at,
            public_url=public_url,
            halt=True,
            reason="unsupported_rail",
        )

    fingerprint = transfer_fingerprint(rail=rail, transfer=transfer)
    decision, signals = _policy_signals(rail=rail, transfer=transfer, mandate=mandate, context=context)

    hop_meta = None
    if fuse_id and fuse_hop:
        hop_meta = fuse_hop(fuse_id)
        if isinstance(hop_meta, dict):
            if hop_meta.get("halt") or hop_meta.get("verdict") is False or hop_meta.get("state") == "DEAD":
                decision = "NO_GO"
                if "fuse_dead" not in signals:
                    signals.append("fuse_dead")

    if signing_required() and not _signing_key():
        decision = "NO_GO"
        signals.append("unsigned_halt")

    receipt = mint_receipt_jwt(

```

```

        evaluation_id=evaluation_id,
        rail=rail,
        decision=decision,
        fingerprint=fingerprint,
        signals=signals,
        agent_id=agent_id,
        ttl_seconds=ttl,
        issuer=public_url.replace("https://", "").replace("http://", "").split("/")[0] or "gate.velaru.xyz",
    )

    if signing_required() and not receipt:
        decision = "NO_GO"
        signals.append("unsigned_halt")

    halt = decision != "GO"
    try:
        from gate import db
    except ImportError:
        import db

    db.record_prefinality_evaluation(
        evaluation_id=evaluation_id,
        account_id=account_id,
        rail=rail,
        decision=decision,
        fingerprint=fingerprint,
        agent_id=agent_id,
        signals=signals,
        receipt_jwt=receipt,
        created_at=created_at,
    )

    return _response(
        evaluation_id=evaluation_id,
        rail=rail,
        decision=decision,
        fingerprint=fingerprint,
        signals=signals,
        receipt=receipt,
        created_at=created_at,
        public_url=public_url,
        halt=halt,
        hop=hop_meta,
        agent_id=agent_id,
        ttl_seconds=ttl,
    )

def _response(
    *,
    evaluation_id: str,
    rail: str,
    decision: str,
    fingerprint: str,
    signals: list[str],
    receipt: str | None,
    created_at: str,
    public_url: str,
    halt: bool,
    reason: str | None = None,
    hop: dict | None = None,
    agent_id: str | None = None,
    ttl_seconds: int = DEFAULT_TTL_SECONDS,
) -> dict:
    base = (public_url or "").rstrip("/")
    out = {
        "spec": SPEC,
        "evaluation_id": evaluation_id,
        "restraint_id": evaluation_id,
        "rail": rail,
        "decision": decision,
        "halt": halt,
    }

```

```

        "signals": signals,
        "fingerprint": fingerprint,
        "receipt": receipt,
        "expires_in": ttl_seconds,
        "created_at": created_at,
        "verify_url": f"{base}/v1/prefinality/verify",
        "manifest": f"{base}/.well-known/prefinality.json",
        "clearance_only": True,
        "write_executed": False,
        "their_production": False,
    }
    if agent_id:
        out["agent_id"] = agent_id
    if reason:
        out["reason"] = reason
    if hop:
        out["fuse_hop"] = {
            "state": hop.get("state"),
            "verdict": hop.get("verdict"),
            "halt": hop.get("halt"),
        }
    if decision == "GO":
        out["message"] = "Pre-finality GO - rail may commit only if receipt verifies and is unexpired."
    elif decision == "HOLD":
        out["message"] = "Pre-finality HOLD - human review required before commit."
    else:
        out["message"] = "Pre-finality NO_GO - do not sign or send. Fail closed."
    return out

def jwks() -> dict:
    pub = _public_key_bytes()
    kid = key_id()
    if not pub or not kid:
        return {"keys": []}
    return {
        "keys": [
            {
                "kty": "OKP",
                "crv": "Ed25519",
                "kid": kid,
                "x": _b64url(pub),
                "use": "sig",
                "alg": "EdDSA",
            }
        ]
    }

def manifest(public_url: str) -> dict:
    base = (public_url or "").rstrip("/")
    return {
        "spec": SPEC,
        "name": "Gate pre-finality clearance",
        "description": (
            "Rail-agnostic GO/NO-GO before irreversible commit. "
            "x402 (agent wallet sign) and RTP/FedNow (instant fiat credit) adapters share one receipt."
        ),
        "rails": [
            {
                "id": "x402",
                "status": "live",
                "hook": "before_wallet_sign",
                "currencies": ["USDC"],
                "evaluate": f"{base}/v1/prefinality/evaluate",
                "demo_evaluate": f"{base}/demo/prefinality/evaluate",
                "sdk": f"{base}/sdk/prefinality/wrap.mjs",
            },
            {
                "id": "rtp",
                "status": "adapter",
                "hook": "before_payment_order",
            }
        ]
    }

```

```

        "note": "FedNow and RTP abstract to payment_type=rtp on PSP APIs (e.g. Modern Treasury).",
        "currencies": ["USD"],
        "evaluate": f"{base}/v1/prefinality/evaluate",
        "verify": f"{base}/v1/prefinality/verify",
    },
],
"evaluate": f"{base}/v1/prefinality/evaluate",
"verify": f"{base}/v1/prefinality/verify",
"jwks": f"{base}/.well-known/prefinality-jwks.json",
"fail_closed": True,
"receipt_ttl_seconds_default": DEFAULT_TTL_SECONDS,
"decisions": list(DECISIONS),
"their_production": False,
}

```